

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Facultad de Ciencias Puras · Carrera de Ingeniería Informática
Potosí — Bolivia

INF 220 — PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

GUÍA DE LABORATORIO N.º 2

Clases y objetos: definición, atributos, métodos e instanciación

Unidad	Unidad 1 — Programación Orientada a Objetos
Tema	Clase vs. objeto · atributos y métodos · instanciación con new · referencias y ciclo de vida del objeto
Requisito	Guía de Laboratorio N.º 1 (entorno instalado y funcionando)
Stack	JDK Temurin 25 (LTS) · Visual Studio Code · Extension Pack for Java
Duración	1 sesión
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

1 Objetivos

► Objetivo general

Diseñar e implementar clases en Java definiendo su estado y comportamiento, para luego crear objetos a partir de ellas y comprender su ciclo de vida en memoria.

► Objetivos específicos

- Distinguir con claridad el concepto de clase del concepto de objeto.
- Definir una clase con atributos (estado) y métodos (comportamiento).
- Instanciar objetos con el operador new y acceder a sus miembros.
- Implementar métodos que reciben parámetros y devuelven valores.
- Comprender qué es una referencia y cómo se administra el ciclo de vida de un objeto.

2 Requisitos previos

Software	JDK Temurin 25 + Visual Studio Code con Extension Pack for Java (instalados en la Guía 1).
Conocimientos	Estructura de un programa Java, tipos de datos, variables, operadores y control de flujo (Guía 1).

3 Marco teórico

► Clase y objeto: el molde y la pieza

Una **clase** es una plantilla o molde que describe cómo serán los objetos de un cierto tipo: qué datos guardan y qué acciones pueden realizar. Un **objeto** es una instancia concreta creada a partir de esa clase. La clase Estudiante es el molde; «Ana, 20 años» y «Luis, 22 años» son dos objetos distintos hechos con ese molde.

DEFINICIÓN • Los dos componentes de una clase

Atributos (campos) — describen el *estado*: los datos que guarda cada objeto (nombre, edad, promedio).

Métodos — describen el *comportamiento*: las acciones que el objeto puede realizar (mostrar sus datos, decir si aprobó).

► Instanciación: crear objetos con new

Definir la clase no crea ningún objeto; solo describe el molde. Para obtener un objeto real se usa el operador **new**, que reserva memoria y devuelve una **referencia** hacia ese objeto:

```
Estudiante e1 = new Estudiante(); // e1 es una referencia al objeto
e1.nombre = "Ana";                // se accede a un atributo con el punto
```

NOTA • Referencia vs. objeto

La variable `e1` no «contiene» al estudiante: guarda una *referencia* (una dirección) hacia el objeto que vive en memoria. Si copias `e1` en otra variable, ambas apuntan al mismo objeto.

► La palabra clave this

Dentro de un método, **this** hace referencia al objeto sobre el cual se llamó el método. Sirve para distinguir un atributo de un parámetro con el mismo nombre, y para dejar claro que se opera sobre el propio objeto.

► Ciclo de vida de un objeto

- **Creación** — con **new** se reserva memoria y el objeto queda listo para usarse.
- **Uso** — se leen y modifican sus atributos y se invocan sus métodos mientras exista alguna referencia hacia él.
- **Destrucción** — cuando ya no queda ninguna referencia apuntándolo, el objeto queda inaccesible y el *recolector de basura* (garbage collector) de la JVM libera su memoria automáticamente.

NOTA

A diferencia de otros lenguajes, en Java no se libera la memoria a mano: la JVM lo hace por vos cuando el objeto deja de ser alcanzable.

4 Desarrollo de la práctica

Trabajaremos con dos archivos dentro de una carpeta `Lab02`: la clase `Estudiante.java` (el molde) y `Principal.java` (con el método `main` que crea y usa los objetos).

► Actividad 1 — Definir la clase con atributos

Crea el archivo `Estudiante.java` con tres atributos que representen el estado de un estudiante:

```
public class Estudiante {
    String nombre;
    int edad;
    double promedio;
}
```

NOTA

Por ahora los atributos quedan accesibles directamente; en la Guía 3 aprenderemos a protegerlos con encapsulamiento (`private`) y constructores.

► Actividad 2 — Instanciar un objeto y usar sus atributos

Crea el archivo `Principal.java`. Instancia un estudiante, asigna sus datos y muéstralos:

```
public class Principal {
    public static void main(String[] args) {
        Estudiante e1 = new Estudiante();
        e1.nombre = "Ana";
        e1.edad = 20;
        e1.promedio = 78.5;

        System.out.println("Nombre: " + e1.nombre);
        System.out.println("Edad: " + e1.edad);
        System.out.println("Promedio: " + e1.promedio);
    }
}
```

1. Ejecuta `Principal.java` con el botón Run (▶) y verifica la salida en la consola.

► Actividad 3 — Agregar comportamiento (métodos)

Vuelve a `Estudiante.java` y añade dos métodos: uno que muestre los datos y otro que indique si el estudiante aprobó:

```
public class Estudiante {
    String nombre;
    int edad;
    double promedio;

    void mostrarDatos() {
        System.out.println(nombre + " (" + edad + " años) - promedio: " + promedio);
    }

    boolean aprobo() {
        return promedio >= 51;
    }
}
```

Ahora usa esos métodos desde `Principal.java`:

```
e1.mostrarDatos();
if (e1.aprobo()) {
    System.out.println("El estudiante aprobó.");
}
```

```

} else {
    System.out.println("El estudiante reprobó.");
}

```

EJEMPLO • Observa

mostrarDatos() no devuelve nada (void), solo actúa; en cambio aprobo() devuelve un boolean que se usa dentro del if.

► Actividad 4 — Métodos con parámetros y retorno

Añade a Estudiante.java un método sin parámetros que incremente la edad, y otro que reciba un parámetro y devuelva el nuevo promedio:

```

void cumplirAnios() {
    this.edad = this.edad + 1;
}

double subirPromedio(double puntos) {
    this.promedio = this.promedio + puntos;
    return this.promedio;
}

```

Pruébalos desde Principal.java:

```

e1.cumplirAnios();
double nuevo = e1.subirPromedio(5.5);
System.out.println("Nueva edad: " + e1.edad);
System.out.println("Nuevo promedio: " + nuevo);

```

► Actividad 5 — Varios objetos con un arreglo

Crea tres estudiantes, guárdalos en un arreglo de objetos y recórrelos con un bucle mostrando sus datos:

```

Estudiante[] curso = new Estudiante[3];

curso[0] = new Estudiante();
curso[0].nombre = "Ana"; curso[0].edad = 20; curso[0].promedio = 78.5;
curso[1] = new Estudiante();
curso[1].nombre = "Luis"; curso[1].edad = 22; curso[1].promedio = 45.0;
curso[2] = new Estudiante();
curso[2].nombre = "Marta"; curso[2].edad = 19; curso[2].promedio = 90.0;

for (int i = 0; i < curso.length; i++) {
    curso[i].mostrarDatos();
}

```

IMPORTANTE • Cuidado con las referencias nulas

Un arreglo de objetos recién creado con new Estudiante[3] contiene 3 referencias null (vacías). Debes instanciar cada objeto con new antes de usarlo, o Java lanzará un NullPointerException.

5 Ejercicios propuestos

Resuelve en archivos java independientes y adjúntalos al informe de la práctica:

1. Crea una clase `Rectangulo` con atributos `base` y `altura`, y métodos `area()` y `perimetro()` que devuelvan los valores calculados.
2. Crea una clase `CuentaBancaria` con atributo `saldo` y métodos `depositar(double monto)` y `retirar(double monto)` que actualicen el saldo (el retiro no debe dejar el saldo negativo).
3. Crea una clase `Circulo` con atributo `radio` y un método `area()` que devuelva $\pi \cdot r^2$. Instancia dos círculos distintos y muestra sus áreas.
4. Crea una clase `Producto` con nombre, precio y cantidad, y un método `valorTotal()`. Crea un arreglo de 4 productos y muestra el valor total del inventario.
5. Amplía la clase `Estudiante` con un método `estaEnRiesgo()` que devuelva `true` si el promedio es menor a 51. Recorre el arreglo del curso e imprime solo los nombres de los estudiantes en riesgo.

6 Criterios de evaluación

Criterio	Descripción	Punt.
Definición de clases	Las clases se declaran correctamente con sus atributos y métodos.	25
Instanciación y uso	Los objetos se crean con <code>new</code> y se accede correctamente a sus miembros.	25
Métodos	Los métodos con parámetros y retorno funcionan como se espera.	20
Ejercicios propuestos	Resolución correcta de los ejercicios planteados.	20
Buenas prácticas	Nombres claros, indentación adecuada y código comentado.	10

Total: 100 puntos.

7 Bibliografía y recursos

- Deitel, P. & Deitel, H. — Cómo programar en Java. Pearson.
- Sierra, K. & Bates, B. — Head First Java. O'Reilly.
- Clases y objetos (tutorial oficial): <https://docs.oracle.com/javase/tutorial/java/javaOO/>